

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

13. Listas intensionales

- Inspiradas en la notación matemática para conjuntos:

- $\{x^2 \mid x \in \{1, \dots, 100\} \wedge x \text{ es par}\}$
- `[ x^2 | x <- [1..100], even x ]`

```
unosCuadrados :: [Int]
unosCuadrados = [ x^2 | x <- [1, 2, 3, 4] ]  -- [1, 4, 9, 16]
```

- Las expresiones como `x <- [1, 2, 3, 4]` se llaman *generadores*.

```
divisores :: Integral a => a -> [a]
divisores n = [ d | d <- [1..n], n `mod` d == 0 ]
```

- Las expresiones como `n `mod` d == 0` se llaman *restricciones* (*guardas*, *condiciones*).
- `[ f x | x <- xs, p x ]` es equivalente a `map f (filter p xs)`.

```
dadosQueSuman :: Int -> [(Int, Int)]
dadosQueSuman n = [ (x, y) | x <- [1..6], y <- [x..6], x + y == n ]
```

- Puede haber varios generadores y varias restricciones:
 - se procesan de izquierda a derecha.

```
parejas :: [(Int, Int)]
parejas = [ (x, y) | x <- [1..3], y <- [x..3] ]
           -- [(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)]
```

```
aplanar :: [[a]] -> [a]
aplanar xss = [ x | xs <- xss, x <- xs ]  -- = concat
```

```
esPrimo :: Integral a => a -> Bool
esPrimo n = divisores n == [1, n]
```

- La comparación `divisores n == [1, n]` es más eficiente de lo que parece, gracias a la evaluación perezosa de la lista intensional.

```
losPrimos :: Integral a => [a]
losPrimos = [ n | n <- [2..], esPrimo n ]
```

```

qsort :: Ord a => [a] -> [a]
qsort [] = []
qsort (x:xs) = qsort [ y | y <- xs, y <= x ]
                ++ [x]
                ++ qsort [ y | y <- xs, y > x ]

encuentra :: Eq a => a -> [(a, b)] -> [b]
encuentra k kvs = [ v | (k',v) <- kvs, k == k' ]
-- encuentra 'b' [('a',1), ('b',2), ('c',3), ('b',4)] -- [2, 4]

```

- Se permiten patrones a la izquierda de <-.
 – Los elementos de la lista que no se ajustan al patrón se desprecian.
 – `[ v | ('b', v) <- [('a',1), ('b',2), ('c',3), ('b',4)] ] ~> [2, 4]`.

```

mejor, peor :: [(Int, Int)]
mejor = [ (x, y) | x <- [1..4], even x, y <- [x+1..4], odd y ] -- [(2,3)]
peor = [ (x, y) | x <- [1..4], y <- [x+1..4], even x, odd y ] -- [(2,3)]

```

- Los generadores y las restricciones se pueden mezclar:
 – se siguen procesando de izquierda a derecha.

```

ordenadas, desordenadas :: [(Int, Int)]
ordenadas = [ (x, y) | x <- [1, 2, 3], y <- [4, 5] ]
-- [(1,4), (1,5), (2,4), (2,5), (3,4), (3,5)]
desordenadas = [ (x, y) | y <- [4, 5], x <- [1, 2, 3] ]
-- [(1,4), (2,4), (3,4), (1,5), (2,5), (3,5)]

```

- El orden entre los generadores importa.

```

soloMay :: String -> String
soloMay st = [ c | c <- st, c `elem` ['A'..'Z'] ]

```

- También vale para cadenas de caracteres.

Ejercicios

1. ¿Qué calcula esta expresión: `sum [ 1 | _ <- xs ]`?
2. Escribe una función `gemelos` que genera la lista de pares de primos gemelos, es decir, los pares de la forma $(n, n + 2)$ donde ambos componentes son primos.
3. Escribe una función `cercanos` que genera la lista de pares de primos que se diferencian como máximo en una cantidad dada. Por ejemplo, `take 6 (cercanos 4)` resulta en `[(2, 3), (2, 5), (3, 5), (3, 7), (5, 7), (7, 11)]`.
4. Escribe una función `posiciones` que devuelve una lista con los índices en los que aparece un elemento dado en una lista dada: `posiciones 0 [1, 0, 1, 7, 0] ~> [1, 4]`. Encuentra la forma de hacerlo usando listas intensionales.